COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Sep 9, 2026	**DATE SUBMITTED:**	Sep 13, 2026
NAME:	Bal, Kevin Ranz J.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

What procedures did you perform?

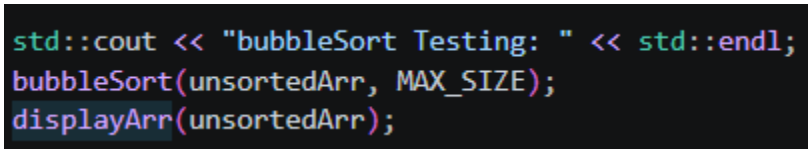
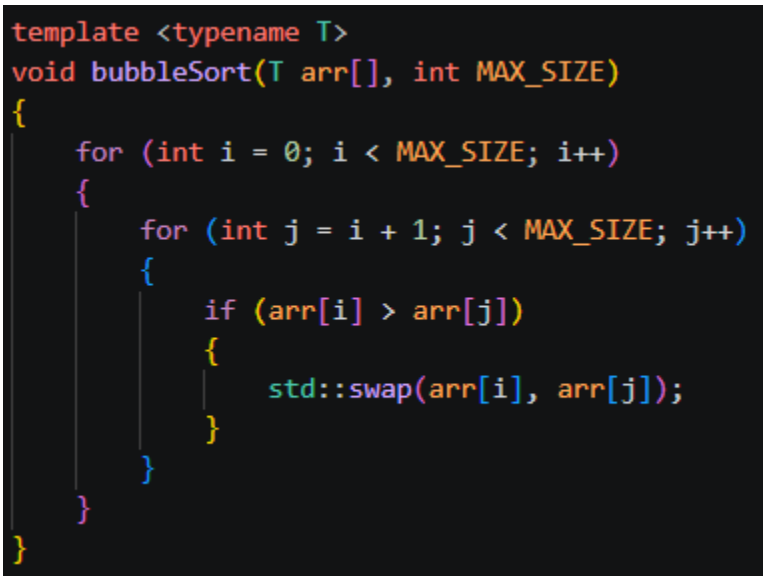
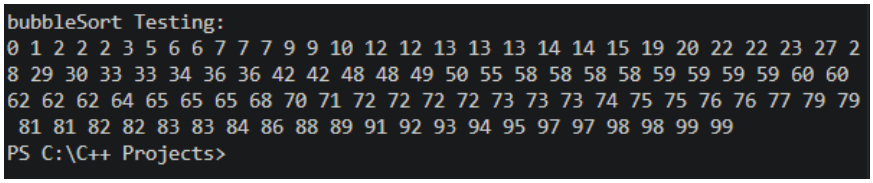
PROCEDURE:
Procedure that performed:
ILO A
Task Title & Task Objective
Table 7-1. Array of Values for Sort Algorithm Testing

Screenshot of the code	CPE21S4_BAL > Midterm > Hands-on Activity > HoA 7.1 > ILO A > main.cpp > ...
	1 #include <iostream>
	2 #include <cstdlib> //for generating random integers
	3 #include <time.h> //will be used for searching function
	4 #include "sorting.h"
	5
	6 const int MAX_SIZE = 100;
	7
	8 int arr[MAX_SIZE];
	9
	10 void displayArr(int arr[]);
	11
	12 int main()
	13 {
	14 std::cout << "Sorting Algorithms Testing: " << std::endl;
	15 // create an array with unsorted list with 100 maximum size
	16 int unsortedArr[MAX_SIZE];
	17 // use the time (0) as seed for random values
	18 srand(time(0));
	19 // populate the array with random random values
	20 for (int i = 0; i < MAX_SIZE; i++)
	21 {
	22 unsortedArr[i] = rand() % 100;
	23 }
	24 displayArr(unsortedArr);
	25
	26 }
	27 void displayArr(int arr[])
	28 {
	29 for (int i = 0; i < MAX_SIZE; i++)
	30 {
	31 std::cout << arr[i] << " ";
	32 }
	33 std::cout << std::endl;
	34 }

Output of the code	Sorting Algorithms Testing: 84 17 24 18 13 59 15 66 14 33 25 21 77 72 81 76 88 21 63 95 51 94 64 51 71 76 94 31 78 85 75 74 20 91 53 13 96 18 82 10 23 15 9 92 0 24 1 47 42 70 48 26 58 48 6 55 47 58 47 87 58 53 82 79 49 53 96 90 65 97 21 32 20 2 89 87 44 67 29 72 89 86 31 27 15 6 63 54 87 64 0 7 7 16 15 5 69 34 37 2 1 PS C:\C++ Projects>
Observation	This code creates and prints an unsorted array with 100 elements. It uses a for loop to fill each position in the array with a random number between 0 and 99. The srand uses the time as a seed, which means the array will look completely different every time it runs the program and another loop reads and prints the values to the console.

ILO A.1

Task Title & Task Objective
Table 7-2. Bubble Sort Technique

Screenshot of the code	
Screenshot of the header code	
Output of the code	
Observation	In this code, it organizes the numbers from smallest to largest. It does this by putting one for-loop inside another for-loop. The outer loop tracks how many times it goes through the list, while the inner loop looks at two numbers next to each other and swaps them if the first number is bigger. Because it runs a loop inside a loop, it can be quite slow on big lists.

PROCEDURE:

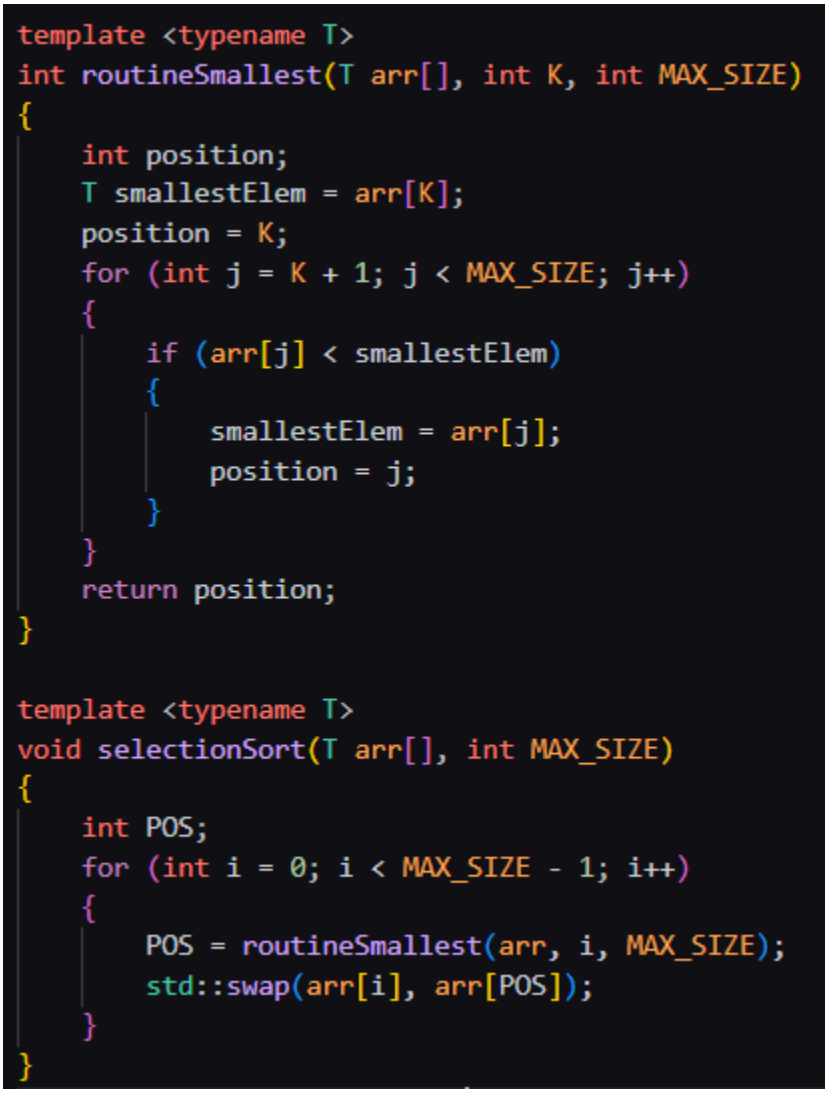
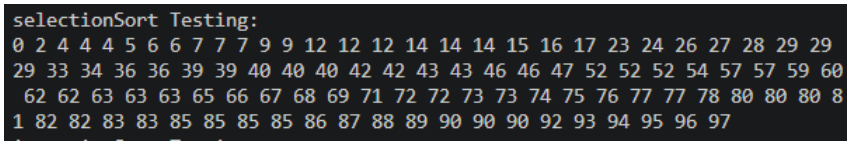
Procedure that performed:

ILO A.2

Task Title & Task Objective
Table 7-3. Selection Sort Algorithm

Screenshot of the code	

```
std::cout << "selectionSort Testing: " << std::endl;
selectionSort(unsortedArr, MAX_SIZE);
displayArr(unsortedArr);
```

Screenshot of the header code	
Output of the code	
Observation	In this code, it sorts the array by finding the smallest number in the unsorted part and swapping it with the first unsorted position. In each turn of the loop, it calls the helper function that is created to look through the remaining numbers and find the smallest one. After finding it, the code uses a swap function to change their positions. Because the helper function runs its own loop inside the main loop.

```
template <typename T>
int routineSmallest(T arr[], int K, int MAX_SIZE)
{
    int position;
    T smallestElem = arr[K];
    position = K;
    for (int j = K + 1; j < MAX_SIZE; j++)
    {
        if (arr[j] < smallestElem)
        {
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], int MAX_SIZE)
{
    int POS;
    for (int i = 0; i < MAX_SIZE - 1; i++)
    {
        POS = routineSmallest(arr, i, MAX_SIZE);
        std::swap(arr[i], arr[POS]);
    }
}
```

```
selectionSort Testing:
0 2 4 4 4 5 6 6 7 7 7 9 9 12 12 12 14 14 14 15 16 17 23 24 26 27 28 29 29
29 33 34 36 36 39 39 40 40 40 42 42 43 43 46 46 47 52 52 52 54 57 57 59 60
 62 62 63 63 63 65 66 67 68 69 71 72 72 73 73 74 75 76 77 77 78 80 80 80 8
1 82 82 83 83 85 85 85 85 86 87 88 89 90 90 90 92 93 94 95 96 97
```

ILO A.3

Task Title & Task Objective
Table 7-4. Insertion Sort Algorithm [While Loop]

Screenshot of the code	`std::cout << "insertionSort Testing: " << std::endl;`
	`insertionSort(unsortedArr, MAX_SIZE);`
	`displayArr(unsortedArr);`

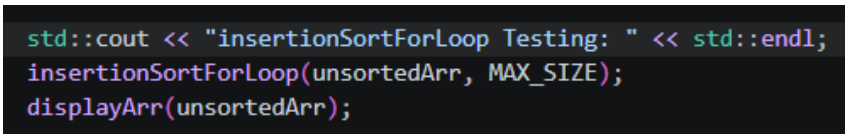
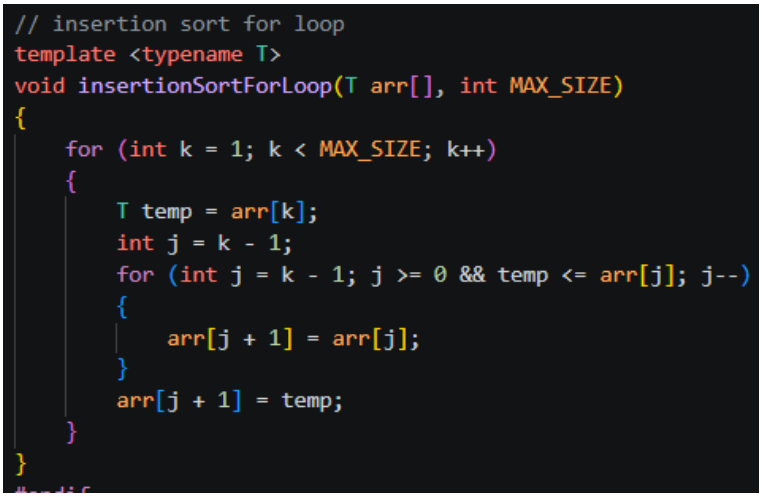
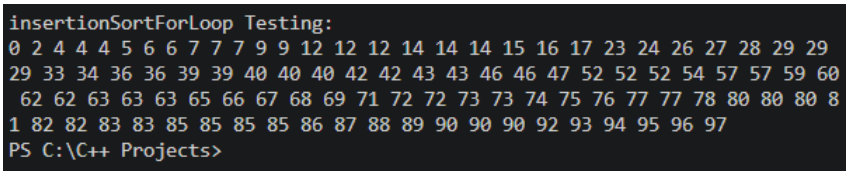
Screenshot of the header code	`// insertion sort while loop`
	`template <typename T>`
	`void insertionSort(T arr[], int MAX_SIZE)`
	`{`
	`  int k = 1, j, temp;`
	`  while (k < MAX_SIZE)`
	`  {`
	`  temp = arr[k];`
	`  j = k - 1;`
	`  while (temp <= arr[j])`
	`  {`
	`  arr[j + 1] = arr[j];`
	`  j--;`
	`  }`
	`  arr[j + 1] = temp;`
	`  k++;`
	`  }`
	`}`

Output of the code	`insertionSort Testing:`
	`0 2 4 4 4 5 6 6 7 7 7 9 9 12 12 12 14 14 14 15 16 17 23 24 26 27 28 29 29`
	`29 33 34 36 36 39 39 40 40 40 42 42 43 43 46 46 47 52 52 52 54 57 57 59 60`
	`62 62 63 63 63 65 66 67 68 69 71 72 72 73 73 74 75 76 77 77 78 80 80 80 8`
	`1 82 82 83 83 85 85 85 85 86 87 88 89 90 90 90 92 93 94 95 96 97`

Observation	In this program, it sorts and uses a main while loop to grab the numbers one by one. The inner while loop looks backward at the numbers that are already sorted. It pushes the bigger numbers to the right to make a blank space and put the current number in. Since there is a loop inside a loop, it can take a long time to run if the list is not sorted or looks messy.

ILO A.3.1

Task Title & Task Objective
Table 7-4. Insertion Sort Algorithm [For Loop]

Screenshot of the code	
Screenshot of the header code	
Output of the code	
Observation	This program does the exact same as the previous one (While Loop Insertion), but it uses for loops instead of while loops. The outer for loop picks the next number to sort, and the inner for loop pushes bigger numbers out of the way to put it in the right position and since it still uses a loop inside a loop, it takes about the same amount of time to run.

```
std::cout << "insertionSortForLoop Testing: " << std::endl;
insertionSortForLoop(unsortedArr, MAX_SIZE);
displayArr(unsortedArr);
```

```
// insertion sort for loop
template <typename T>
void insertionSortForLoop(T arr[], int MAX_SIZE)
{
    for (int k = 1; k < MAX_SIZE; k++)
    {
        T temp = arr[k];
        int j = k - 1;
        for (int j = k - 1; j >= 0 && temp <= arr[j]; j--)
        {
            arr[j + 1] = arr[j];
        }
        arr[j + 1] = temp;
    }
}
```

```
insertionSortForLoop Testing:
0 2 4 4 4 5 6 6 7 7 7 9 9 12 12 12 14 14 14 15 16 17 23 24 26 27 28 29 29
29 33 34 36 36 39 39 40 40 40 42 42 43 43 46 46 47 52 52 52 54 57 57 59 60
 62 62 63 63 63 65 66 67 68 69 71 72 72 73 73 74 75 76 77 77 78 80 80 80 8
1 82 82 83 83 85 85 85 85 86 87 88 89 90 90 90 92 93 94 95 96 97
PS C:\C++ Projects>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

SUPPLEMENTARY:

ILO C

Task Title & Objective:

Pseudocode Algorithm	FUNCTION MAIN START DECLARE unsortedArr[MAX_SIZE] DECLARE candidates[5] DECLARE votesArray[5] DECLARE winner SET candidates = ["Bo Dalton Capistrano", "Cornelius Raymon Agustin", "Deja Jayla Bañaga", "Laila Brielle Yabut", "Franklin Relano Castro"] CALL srand(1024) LOOP i FROM 0 TO MAX_SIZE - 1 unsortedArr[i] = (RANDOM % 5) + 1 END LOOP PRINT "unsortedArr:" CALL displayArr(unsortedArr) PRINT "Selection sort:" CALL selectionSort(unsortedArr, MAX_SIZE) CALL displayArr(unsortedArr) CALL countVotes(unsortedArr, votesArray) PRINT "Vote Results:" LOOP i FROM 0 TO 4 PRINT candidates[i], votesArray[i] END LOOP SET winner = 0 LOOP i FROM 1 TO 4 IF votesArray[i] > votesArray[winner] THEN SET winner = i

```
      END IF
   END LOOP

   PRINT "winner:", candidates[winner], votesArray[winner]
END

FUNCTION displayArr(arr)
START
   LOOP i FROM 0 TO MAX_SIZE - 1
      PRINT arr[i]
   END LOOP
   PRINT newline
END

FUNCTION selectionSort(arr, MAX_SIZE)
START
   LOOP i FROM 0 TO MAX_SIZE - 2
      SET POS = i
      LOOP j FROM (i + 1) TO MAX_SIZE - 1
         IF arr[j] < arr[POS] THEN
            SET POS = j
         END IF
      END LOOP
      CALL swap(arr[i], arr[POS])
   END LOOP
END

FUNCTION countVotes(arr, votes)
START
   LOOP i FROM 0 TO 4
      votes[i] = 0
   END LOOP

   LOOP i FROM 0 TO MAX_SIZE - 1
      INCREMENT votes[arr[i] - 1]
   END LOOP
END
```

Screenshot of the Main Driver Code

```cpp
#include <iostream>
#include <cstdlib> // for generating random integers
#include <string>
#include "supp.h"

const int MAX_SIZE = 100;

void displayArr(int arr[]);
void countVotes(int arr[], int votes[]);

int main()
{
    int unsortedArr[MAX_SIZE];
    std::string candidates[5] = {
        "Candidate 1: Bo Dalton Capistrano",
        "Candidate 2: Cornelius Raymon Agustin",
        "Candidate 3: Deja Jayla Bañaga",
        "Candidate 4: Laila Brielle Yabut",
        "Candidate 5: Franklin Relano Castro"};

    // predetermined seed to make random numbers
    srand(1024);

    // generate a random votes with the range of 1 to 5
    for (int i = 0; i < MAX_SIZE; i++)
    {
        unsortedArr[i] = rand() % 5 + 1;
    }

    std::cout << "unsortedArr: \n";
    displayArr(unsortedArr);

    std::cout << "\nSelection sort: \n";
    selectionSort(unsortedArr, MAX_SIZE);
    displayArr(unsortedArr);

    int votes[5] = {};
    countVotes(unsortedArr, votes); // to count how many votes of each the candidate got

    std::cout << "\nVote Results: \n";
    for (int i = 0; i < 5; i++)
    {
        std::cout << candidates[i] << ": " << votes[i] << " votes\n";
    }
```

```
    int winner = 0;
    for (int i = 1; i < 5; i++)
    {
        if (votes[i] > votes[winner])
        {
            winner = i; // update the winner index if another candidate has more votes
        }
    }

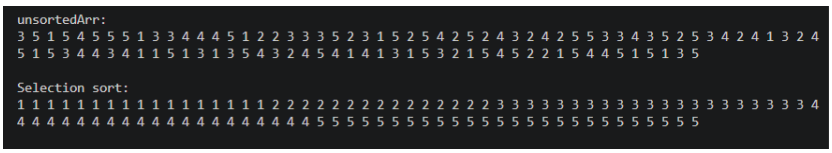
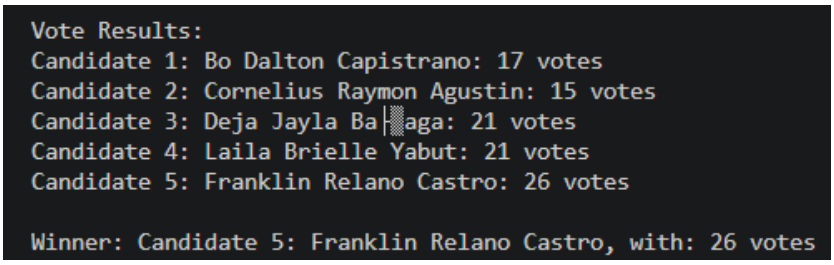
    std::cout << "\nWinner: " << candidates[winner] << ", with: " << votes[winner] << " votes\n";
    return 0;
}

void displayArr(int arr[])
{
    for (int i = 0; i < MAX_SIZE; i++)
    {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

void countVotes(int arr[], int votes[])
{
    for (int i = 0; i < 5; i++)
    {
        votes[i] = 0; // to clear the vote counts and reset it to zero
    }
    for (int i = 0; i < MAX_SIZE; i++)
    {
        votes[arr[i] - 1]++; // increment a vote to the chosen candidate
    }
}
```

Screenshot of the Header Code

```
1  #ifndef SUPP_H
2  #define SUPP_H
3  #include <iostream>
4
5  template <typename T>
6  void selectionSort(T arr[], int MAX_SIZE)
7  {
8      for (int i = 0; i < MAX_SIZE - 1; i++)
9      {
10         int POS = i; // assume the current index position has the smallest number / value
11         for (int j = i + 1; j < MAX_SIZE; j++)
12         {
13             if (arr[j] < arr[POS])
14             {
15                 POS = j; // update the index position if a smaller number is found
16             }
17         }
18         std::swap(arr[i], arr[POS]); // swap the smallest number into the correct index position
19     }
20 }
21 #endif
```

Displaying the sorted array	 Manual Count | Count Result of Algorithm 17 | 17 15 | 15 21 | 21 21 | 21 26 | 26
Output Testing	
Justify why you chose to use this sorting algorithm.	I used selection sort because the problem has a small range of vote options from 1 to 5. Selection sort is straightforward to write and works well for grouping small sets of data. Using this algorithm let me group all of the identical votes together in the array before the counting code runs. This setup makes sure that the counting program can easily go through the list and find the correct total for each candidate.
Question: Was your developed vote counting algorithm effective? Why or why not?	The vote counting algorithm I did was effective and working because it properly matched the manual count results and identified the winning candidate. It gets its goal through the use of two loops. The first loop resets all vote counts to zero, allowing the program to start counting from the start. The second loop iterates through the array that was sorted using the selection sort algorithm, updating the vote totals for each candidate without removing any numbers.
Analysis of the procedure	In this procedure, I analyzed how bubble sort, selection sort, and insertion sort work on an unsorted array. I observed that each method relies on a loop inside another loop to handle searching and swapping numbers. Because of

	this nested structure, I found that all three algorithms have an average and worst-case time complexity of $O(N^2)$. Even though selection sort splits its logic by putting the inner loop into a separate function, it still runs with the same $O(N^2)$ behavior.
Analysis of the supplementary	In the supplementary activity, I applied these sorting concepts to a practical vote counter scenario. I used selection sort to organize a messy list of 100 random votes so that all matching choices were grouped together. Once the array was in order, I implemented a counting algorithm with two loops to tally up the vote counts for each candidate and display the correct final score votes.
Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?	I think I did well in this activity by reusing and applying the previous procedure in this laboratory, searching, and watching videos of how each sorting works and knowing their differences too to finish this activity. It also helped me to visualize how each of them moves numbers around in an array. Even though it took me a while to submit this, I learned how they work and when to use them that I can apply and use in the future. For my areas for improvement, I want to practice and learn to write clean code with fewer lines. I would also like to learn about faster sorting algorithms that do not use nested loops so I can handle larger lists of data.

Conclusion
In this laboratory activity, I learned and successfully showed to me how bubble, selection, and insertion sort operate using array structures. Every algorithm has its own specific way of changing the positions of elements, but they all rely on nested loops that make them slow on large datasets. By creating the vote counter activity, I learned how to apply these concepts to a practical problem and saw that organizing a list first makes it much simpler to count and double-check the final numbers of votes.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chrome AI Search**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explaining the concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **To explain clearly of how I could understand easily the three sorting algorithms and asking to illustrate them where I could visualize how they move and work**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.